

Simple Sorts

Lec 08a Bubble Sort

Overview

Consider an array of integers, and the goal is to sort into non-decreasing order (largest on the right)

We will consider “simple sorts”:

1. Bubble sort
2. Selection sort
3. Insertion sort

We will particularly consider

- “Stability”
- Recurrence structure
- Efficiency (in terms of Big-Oh)
- “Adaptive”?

Bubble sort: Structure

- Outer loop:
 - Repeated scans through array
- Inner loop: on each scan do comparison with immediate neighbour
 - think of air bubbles rising in water
 - do swaps to make sure that the largest number “bubbles up” to the end of the array
- After each inner loop the size of the next scan be decremented as do not need to look again at the end element
 - The outer loop keeps track of the reductions in the range of the inner loop

Pseudo-code

A sketch of the code structure is:

```
sort(int[] A)
    int n = A.size
    for (int k = n-1 ; k > 0 ; k-- ) // outer loop
        for (int j = 1 ; j <= k ; j++ ) // inner loop
            if ( A[j] < A[j-1] )
                // swap A[j] and A[j-1]
                {int tmp=A[j];A[j]=A[j-1];A[j-1]=tmp;}
```

Offline (aside, for discussions):

Could you implement a swap method/routine “swap(j,j-1)” in Java?

Does Java have “pass by reference”?

Can you do it in C++, C#, etc ?

Example – first pass

Suppose $A = [8, 4, 5, 8', 1]$ $n = 5$

- Here $8'$ is just another value 8, but the prime is so we keep track of it – to see where 8 and $8'$ move to
- First pass: $k=n-1=4$, the first few moves are obvious, the 8 is swapped to the right:

8	4	5	8'	1
----------	----------	----------	-----------	----------

4	8	5	8'	1
----------	----------	----------	-----------	----------

4	5	8	8'	1
----------	----------	----------	-----------	----------

- At this point – assuming a reasonable implementation, we do not need a swap of 8 and $8'$, giving

4	5	8	8'	1
----------	----------	----------	-----------	----------

- Then the last swap, of $8'$ and 1, gives:

4	5	8	1	8'
----------	----------	----------	----------	-----------

Bubble sort: Example – second pass

After first pass $A = [4, 5, 8, 1, 8']$

The end element of the array is now no smaller than other element, and so we do not need to look at it again. (We will indicate this with changing the cell background.)

The second pass of the inner loop gives:

4	5	8	1	8'
4	5	8	1	8'
4	5	1	8	8'

Bubble sort: Example –pass 3 & 4

After second pass $A = [4, 5, 1, 8, 8']$

The end two elements are in correct places and so we do not need to look at them again.

The third pass of the inner loop does

4	5	1	8	8'
4	5	1	8	8'
4	1	5	8	8'

The last pass of the inner loop does a final swap

1	4	5	8	8'
---	---	---	---	----

Bubble sort: Stability

We started with $A = [8, 4, 5, 8', 1]$
and finished with $A = [1, 4, 5, 8, 8']$

- As expected, the final array is sorted.
- Also notice that the order of the “duplicate” elements 8 and 8' has not been changed:

Before the sort: 8 is before 8'

After the sort: 8 is before 8'

- Sorting methods that preserve the order of duplicates are said to be “stable”
- **Bubble sort is a stable sorting algorithm**
 - (assuming a reasonable implementation that only swaps elements that must be swapped)

“Stability” : Why do we care !?

- If sorting integers, `int[]`, then it does not really matter if identical integers are swapped
 - Why care about stability at all ???

VITAL NOTE:

“stable” does NOT mean “the code does not break” !!

This is a common error.

“Stability” : General relevance

- Often, we sort objects according to some comparison function:

`compare(o1, o2)` returns negative, zero, or positive for “less than”, “equal”, or “greater than”

- `compare(o1,o2)==0` means objects `o1` and `o2` are
 - equal with respect to the desired ordering
 - but not necessarily that they have the same contents
 - i.e. **being “equal” does not mean identical !**
- E.g. object is a row of a spreadsheet and compare uses just one specified column - many different rows can be equal under the comparison, but not identical

“Stability” : Spreadsheets

- If sorting a spreadsheet, then might sort by one column then another.
- Do not want the sorting to unnecessarily change the order of the rows, as this can be annoying and confusing.
- *"Sort by column A, followed by a stable sort on column B"*
means that still will have a secondary sort on column A
- **EXERCISE (offline): Experiment with Excel to see if it does stable sorts or not.**

Complexity? Comparisons and swaps

- In analysis we do not need to count every single primitive operation, instead just count
 - Usage of the comparison operators
 - Swaps
- These will be sufficient for extracting the Big-Oh behaviours
 - If we did a more detailed count then the results would not change

Pass 1: $n-1$ comparison ; up to $n-1$ swaps

Pass 2: $n-2$ comparison ; up to $n-2$ swaps

...

Pass $n-1$: 1 comparison ; up to 1 swaps

Complexity? Arithmetic series

Pass 1: $n-1$ comparisons ; up to $n-1$ swaps

Pass 2: $n-2$ comparisons ; up to $n-2$ swaps

...

Pass $n-1$: 1 comparisons ; up to 1 swaps

The total comparisons is

$$(n-1) + (n-2) + \dots + 1$$

which is just an arithmetic series (backwards)
(See auxiliary tutorial slides)

Total comparisons = $n(n-1)/2$ which is $\Theta(n^2)$

Complexity? Best and worst cases

- Worst case : the array is reverse ordered
 - every potential swap is needed
 - Numbers of comparisons and swaps are both $\Theta(n^2)$
- Best case: the array is already ordered – no swaps are needed.
 - However, every comparison is still performed
 - Overall complexity is still $\Theta(n^2)$

Adaptive sorting

- In practice we are often sorting arrays that are already “almost sorted”
- If a sorting routine is much faster when the array is “almost sorted” then it is called “adaptive”
 - Because it adapts well to “good inputs”
- Bubble sort is not adaptive – it does not take much advantage of “almost sorted” inputs

Generalise to Simplify

Instead of always doing

```
sort( int[] A )
```

Consider sorting a sub-array defined by the first and last indices

```
sort( int[] A, int L, int R )
```

This seems to make things more complicated but paradoxically makes the implementations much clearer.

It is a standard technique we will use frequently

Recursive version of bubble sort

- Given the structure of bubble sort as doing a scan and then essentially doing bubble sort on the reduced array, we can write it as a recursive algorithm
- Each recursive call will work on a shorter sub-array
- Hence:

```
sort(int[] A, 0, k)
    for (int j = 1 ; j <= k ; j++ )
        if ( A[j] < A[j-1] ) << swap A[j] and A[j-1] >>
    // recurse on smaller sub-array if not finished
    if ( k > 1 )
        sort(int[] A, 0, k-1)
```

- We will use this again later to re-analyse the complexity

Correctness argument?

Basic structure of the argument:

Each inner pass moves one more maximal element to the end of the subarray.

This new end element of subarray cannot be smaller than the previous elements.

Termination is “obvious” as subarrays are strictly smaller on each iteration of the outer loop

Summary of bubble sort

- PRO: Simple and easy-to-implement
- CON: It is $\Theta(n^2)$
 - This is slow – even at $n=10^6$ we would have troubles
- PRO: It is naturally stable
 - No unnecessary re-ordering of elements
- CON: It is not adaptive
 - Unless do modifications, it does not naturally take advantage of the common case if the input already being “nearly sorted”
- PRO: it can also be easily implemented on linked lists (even singly-linked)
 - Exercise (offline): convince yourself of this

Minimum Expectations

- Know the bubble sort algorithm and how it works on examples
- Be able to justify/prove the big-Oh family of behaviours of bubble sort
- Understand the notion of it being a “stable” sort
- Understand the notion of “adaptive” and that it is not adaptive